

ARTI403 – Lab 3

Image Manipulations using OpenCV Library

Name: Ali Jafar Altaweel

ID: 2240006456

Section: 7MA2

GitHub: <https://github.com/Ali-developer/Image-Processing/tree/main/Lab3>

In this lab, I worked with the OpenCV library to read, display, save and convert images, and then applied geometric and intensity transformations to them. The main objective was to understand how a digital image is stored in the computer as an array, and how moving or remapping its values changes what we see.

Lab Tasks

Task 1 – Geometric transformations

Use any image processing library and apply the following geometric transformations over the image:

- Increase the size of the image.
- Rotate the image by 120 degrees.
- Perform Sheer operations over the image.

Task 2 – Intensity transformations

Use any image processing library and apply the following intensity transformations over the image:

- Create a Negative Image.
- Log with (choose an appropriate constant for your image).
- Power law (choose a value for gamma that increases the contrast).

Images Used

I used `input.jpg`, an 812×458 color photograph of a forest stream with moss covered rocks, which is the input image of the OpenCV with Python By Example book that the manual is based on. It is stored in the `images` folder of this lab, together with `output.jpg`, the grayscale copy that the lab itself saves.

The image is quite dark, with a mean gray level of about 90 and almost 78% of the pixels below 128, which turned out to matter for the intensity transformations.

Reading and Displaying an Image

I read the image with `cv2.imread()` and confirmed that OpenCV returns a plain NumPy array of type `uint8` with the shape `(458, 812, 3)`, meaning rows, columns and channels. One thing I had to keep in mind is that OpenCV stores the channels in BGR order, not RGB, so the first pixel was printed as blue, green, red.

The manual displays images with `cv2.imshow()`, which opens an external window and blocks the notebook until a key is pressed. I kept the original lines as comments and wrote a small `show()`

helper that displays the image inline with Matplotlib after converting it from BGR to RGB, so all the results are saved inside the notebook.

Input image



Figure 1 – The input image read with `cv2.imread()`

Grayscale and Saving

I loaded the image directly in grayscale with the `cv2.IMREAD_GRAYSCALE` flag, which left a single channel of shape (458, 812). I then saved it with `cv2.imwrite()` and read it back to check the result. The file was written correctly, but the values differed from the original by up to 3 gray levels, because JPEG is a lossy format.

Grayscale



Figure 2 – The image loaded directly in grayscale

Color Spaces

I converted the color image to grayscale with `cv2.cvtColor()` and listed every conversion flag available. The OpenCV version I used has 374 `COLOR_` flags, which is more than the 190 mentioned in the manual. I also had to fix the manual's print line, because it is written in Python 2 syntax.

I then converted the image to YUV and displayed the three channels separately:

- The Y channel holds the brightness and looks like a normal grayscale image, with a standard deviation of about 55
- The U channel, the blue projection, is almost flat, with values only between 60 and 144
- The V channel, the red projection, is even flatter, between 104 and 159

This shows why YUV was used for television: almost all of the information is in the brightness channel, so the two color channels can be stored with much less precision.

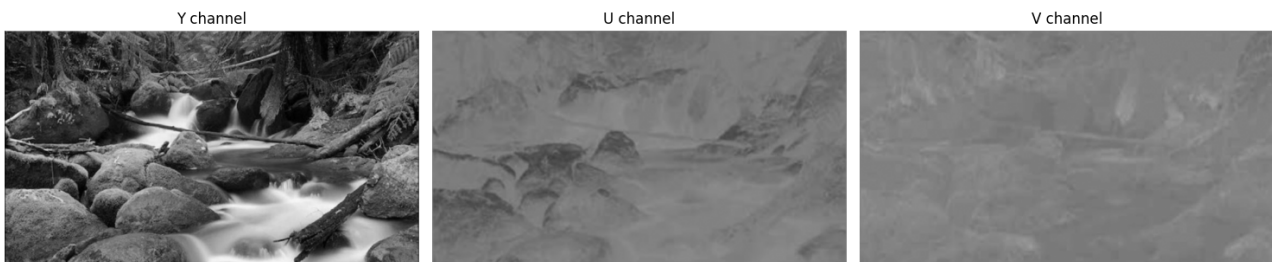


Figure 3 – The Y, U and V channels of the image

Geometric Transformations

For the first task I applied three geometric transformations.

Increasing the size. I scaled the image by 2 in both directions with `cv2.resize()`, which gave a 1624×916 image with 4 times more pixels. I also zoomed on a small crop and compared four interpolation methods. Nearest neighbor produced visible square blocks, while linear, cubic and Lanczos produced smoother results, with cubic and Lanczos keeping the edges the sharpest.

Rotating by 120 degrees. I built the rotation matrix with `cv2.getRotationMatrix2D()` and applied it with `cv2.warpAffine()`. When I kept the original canvas size, the corners of the rotated image were cut off. To keep the whole image I computed the size needed from the sine and cosine of the angle, which gave an 802×932 canvas, and shifted the matrix so the image stayed centered.

Shearing. I wrote the shear matrices myself, one for a horizontal shear where each row is shifted proportionally to its height, and one for a vertical shear. With a factor of 0.4 the rectangle became a parallelogram, and I had to widen the canvas so the shifted part was not lost. Negative factors lean the image the other way.

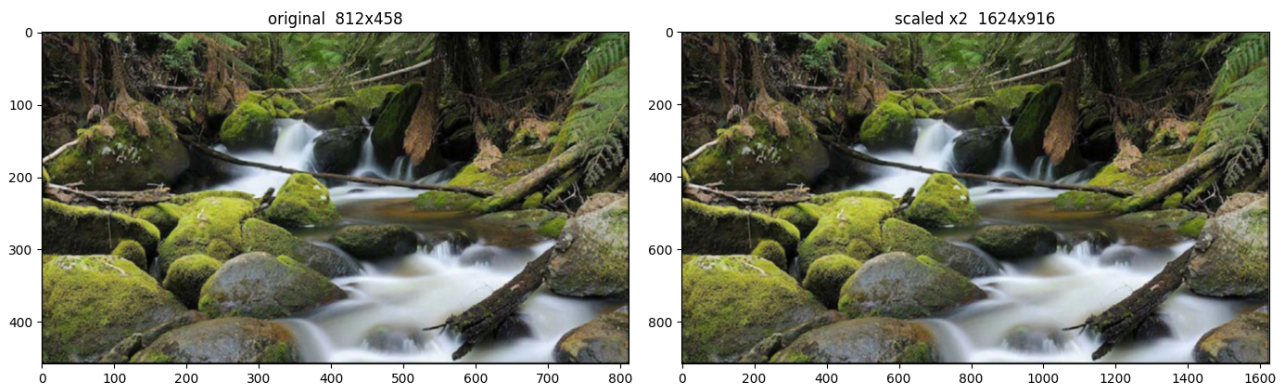


Figure 4 – Task 1: the image scaled by 2



Figure 5 – Task 1: interpolation methods compared on a zoomed crop

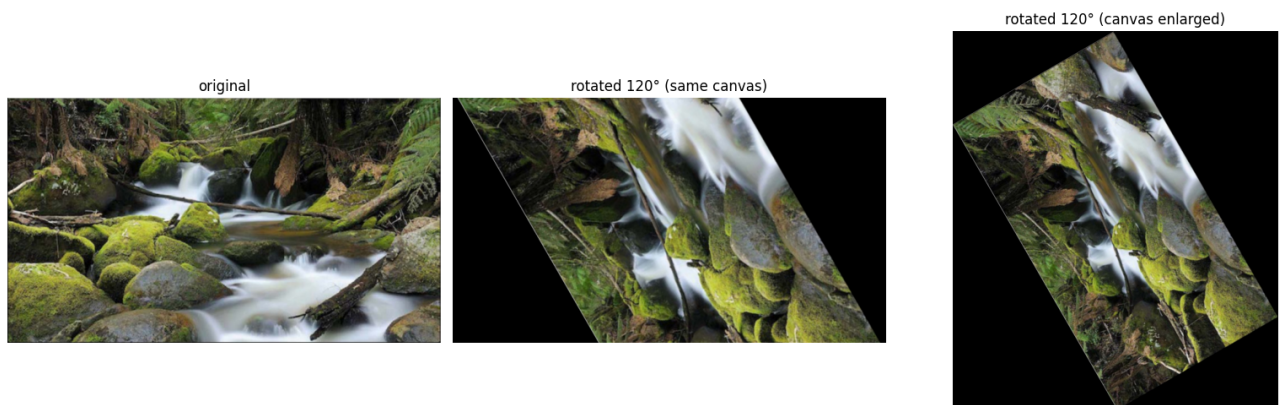


Figure 6 – Task 1: rotation by 120 degrees



Figure 7 – Task 1: horizontal and vertical shear ($sh = 0.4$)

Intensity Transformations

For the second task I worked on the grayscale image in floating point, so that nothing would wrap around.

Negative. I computed 255 minus each pixel. The mean went from 90.3 to 164.7, which adds up to 255, and taking the negative twice gave back the exact original. The dark rocks became bright and the white water became dark.

Log. I chose the constant so that the brightest pixel is mapped exactly to 255, which gave $c = 255 / \log(1 + 253) \approx 46$. The log curve strongly expands the dark values, so the mean rose from 90 to 197 and the details in the shadows became visible. I also tried other constants: with $c = 70$, more than 80% of the pixels were saturated at 255.

Power law. Choosing gamma was the most interesting part of this lab. My first idea was to measure the contrast with the standard deviation of the whole image, but it was highest at gamma = 1, which means that no gamma increases the global contrast of this image. The reason is that every gamma curve stretches one part of the gray range and compresses the other part.

Since 78% of the image is dark, I measured instead the contrast inside the dark region. It rose from 32.7 for the original to 35.8 at gamma = 0.6, so I chose that value. A gamma smaller than 1 has a slope greater than 1 for dark inputs, so it spreads exactly the values where most of the image lies, while gammas larger than 1 made the image darker and flatter.

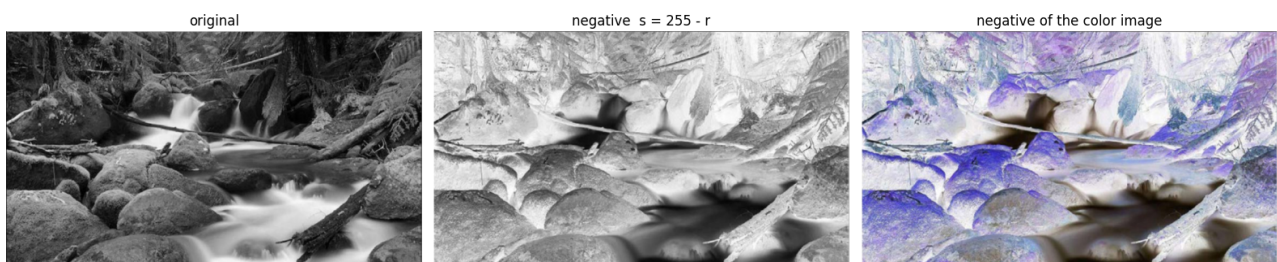


Figure 8 – Task 2: negative image



Figure 9 – Task 2: log transformation with different constants



Figure 10 – Task 2: power law with different gammas

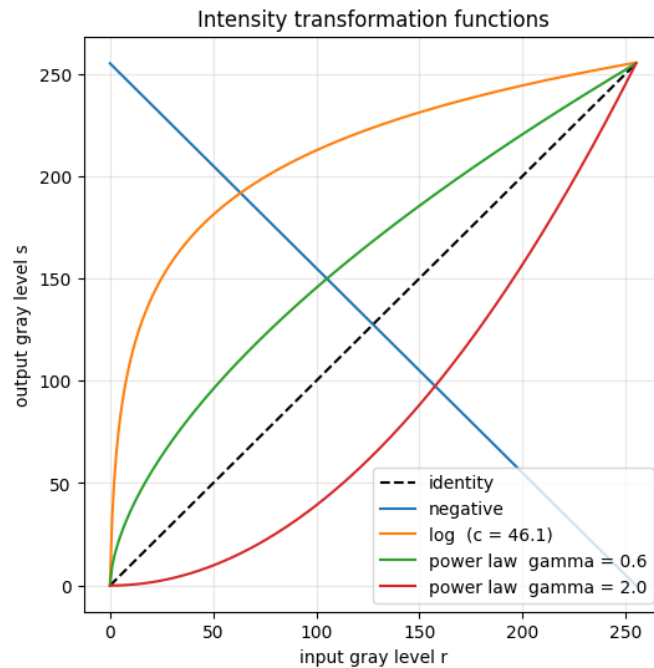


Figure 11 – Task 2: the transformation curves



Figure 12 – Task 2: summary of the three intensity transformations

Observations

The most important observation was that a transformation has to be chosen for a specific image. The log constant depends on the maximum value of the image, and the best gamma depends on where most of the gray levels are, so a value that improves one image can ruin another.

I also noticed that all the geometric transformations are the same operation, a 2×3 affine matrix applied with `warpAffine()`, and that the real difficulty is not the matrix but the output canvas and the interpolation of the new pixels.

Conclusion

This lab showed how OpenCV represents an image as a NumPy array and how to read, convert and save it. Geometric transformations move the pixels to new positions, while intensity transformations keep the positions and change the values.

The negative, log and power law transformations are simple formulas applied to every pixel, but their effect depends entirely on the histogram of the image. For this dark image, the log transformation and a gamma of 0.6 both made the hidden details in the shadows visible.

Submission

The complete work for this lab is in my GitHub repository, in the **Lab3** folder: <https://github.com/Ali-developer/Image-Processing/tree/main/Lab3>

As required, the Lab3 folder contains the following files:

- Jupyter notebook: Lab3.ipynb, executed, with all the outputs and the task solutions
- Images used: images/input.jpg, images/output.jpg
- This report: Lab3_Report.pdf, and the same write-up as README.md